Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Защита веб-периметра и ограничение интенсивности сетевого трафика средствами межсетевых экранов прикладного уровня»

Выполнил:

Студент 4 курса

Группы ИИ-26

Петручик Д.А.

Проверила:

Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение методов развёртывания WAF, написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов на базе обратного прокси-сервера Nginx.

Ход работы

Задание 1. Базовая фильтрация угроз через WAF (OWASP Core Rule Set)
Выполнить отправку вредоносного SQLi-запроса (например, /?id=1' OR '1'='1) напрямую к приложению (http://127.0.0.1:5000) с помощью curl и убедиться, что приложение его обрабатывает (возвращает код 200 OK). Повторить тот же SQLi-запрос через защищённый Nginx-периметр (http://127.0.0.1). Убедиться, что встроенный набор правил OWASP CRS перехватывает атаку, возвращая ошибку 403 Forbidden.
Задание 2. Написание пользовательских правил фильтрации (L7)
Выполнить отправку легитимного HTTP-запроса, подменив заголовок User-Agent на вредоносный сканер уязвимостей из индивидуального варианта, с помощью curl -H "User-Agent: ...". Написать пользовательское правило WAF в подготовленном файле nginx/custom.conf с присвоением уникального ID из варианта. Правило должно блокировать запросы, содержащие данный User-Agent. Перезапустить контейнеры (docker-compose restart waf) и убедиться, что WAF успешно блокирует запрос сканера (код 403).
Задание 3. Защита от L7 DDoS-атак (Rate Limiting)
Выполнить стресс-тест целевого эндпоинта (согласно варианту) утилитой Apache Bench, отправив 500 запросов в 10 потоков: ab -n 500 -c 10 http://127.0.0.1<ваш_эндпоинт>. Зафиксировать, что сервер успешно обработал все запросы без ошибок (Failed requests: 0). Сконфигурировать директивы ограничения интенсивности трафика (limit_req_zone и limit_req) в файле nginx/user_nginx.conf, используя параметры rate и burst из индивидуального варианта. Перезапустить контейнер WAF, повторить стресс-тест утилитой ab и зафиксировать блокировку избыточных запросов (сервер должен начать отдавать код 503 Service Unavailable для запросов, превышающих лимит).
Задание 4. Проектирование профилей защиты (Без написания кода)
Составить конфигурацию директив Nginx для защиты от атаки «Slowloris» (Slow HTTP Attack), указав оптимальные значения таймаутов (client_body_timeout, client_header_timeout) и обосновать свой выбор. Спроектировать архитектурную схему применения CDN (Content Delivery Network) в связке с WAF для защиты от объёмных (L3/L4) DDoS-атак.

Вариант	WAF: Блокируемый User-Agent	ID правила WAF	Rate Limit: Эндпоинт	Rate Limit: rate	Rate Limit: burst
4	Nmap Scripting Engine	10004	/api/password_reset	1r/s	2

Для выполнения лабораторной работы потребовалось установить Docker Desktop: он разворачивает лабораторный стенд из двух контейнеров (WAF и backend) одной командой. Без него запуск ModSecurity на Windows невозможен, так как пришлось бы вручную собирать Nginx, ModSecurity и OWASP CRS под Windows, что значительно усложнило бы

настройку. Вместе с Docker потребовался WSL 2, который обеспечивает Linux-окружение, необходимое для корректной работы Docker Desktop.

Для выполнения нагрузочного тестирования потребовалась утилита hey.exe, которая позволяет отправить 500 запросов в 10 потоков. Это необходимо для проверки работы механизма Rate Limiting, так как в варианте 4 установлен очень низкий лимит — 1 запрос в секунду с burst = 2**.** При обычной отправке запросов через `curl` интенсивность недостаточна для превышения установленного ограничения, поэтому блокировка избыточных запросов с кодом 503 Service Unavailable может не проявиться.

В рамках индивидуального варианта была настроена защита эндпоинта `/api/password_reset` с использованием директив Nginx `limit_req_zone` и `limit_req`. Также было создано пользовательское правило WAF для блокировки подозрительного User-Agent `Nmap Scripting Engine` с идентификатором правила 10004**.**

Задание 1. Базовая фильтрация OWASP CRS

Что делали: проверили работу встроенного набора правил OWASP CRS при обнаружении SQL-инъекции. Сначала запрос был отправлен напрямую к backend-приложению (обход WAF), затем тот же запрос был выполнен через защищённый Nginx-периметр.

1.1 — напрямую к бэкенду (порт 5000)

curl.exe -i http://127.0.0.1:5000/?id=1%27%20OR%20%271%27=%271

1.2 — через WAF-периметр (порт 80)

curl.exe -i http://127.0.0.1/?id=1%27%20OR%20%271%27=%271

Что получили:

Запрос	Код	Значение
Напрямую (5000)	HTTP/1.1 200 OK + Backend OK	Backend доступен напрямую и обрабатывает SQL-инъекцию без фильтрации
Через WAF (80)	HTTP/1.1 403 Forbidden	OWASP CRS обнаружил сигнатуру SQL-инъекции и заблокировал вредоносный запрос

Вывод: Базовый набор правил OWASP CRS успешно обнаруживает и блокирует SQL-инъекции на уровне WAF. Защищённый Nginx-периметр предотвращает передачу опасных запросов к backend-приложению без необходимости добавления пользовательских правил.

Задание 2. Пользовательское правило WAF

Что делали: написали пользовательское правило ModSecurity для блокировки запросов с заданным User-Agent из индивидуального варианта. Для варианта 4 необходимо блокировать сканер Nmap Scripting Engine с идентификатором правила 10004

Что ввели в nginx/custom.conf:

SecRule REQUEST_HEADERS:User-Agent "@contains Nmap Scripting Engine" \

"id:10004,phase:1,deny,status:403,msg:'Blocked Custom Scanner UA - Variant 4'"

Что вводили в терминал:

Проверка ДО правила - запрос проходил
curl.exe -i -H "User-Agent: Nmap Scripting Engine" http://127.0.0.1/

Перезапуск WAF
docker compose restart waf

Проверка ПОСЛЕ правила

блокировка:

curl.exe -i -H "User-Agent: Nmap

Scripting Engine" http://127.0.0.1/

Контроль - другой UA не блокируется
curl.exe -i -H "User-Agent: Mozilla/5.0" http://127.0.0.1/

Что получили:

Запрос	Код	Значение
До правила, UA Nmap Scripting Engine	200 OK	Правило ещё не применялось
После правила, UA Nmap Scripting Engine	403 Forbidden	Правило 10004 сработало
После правила, UA Mozilla/5.0	200 OK	Правило применяется только к заданному User-Agent

Лог WAF подтверждает:
[error] ModSecurity: Access denied with code 403 (phase 1).
[id "10004"]
[msg "Blocked Custom Scanner UA - Variant 4"]
[file ".../custom.conf"]

Вывод: Пользовательское правило ModSecurity с идентификатором 10004 успешно блокирует запросы с User-Agent Nmap Scripting Engine, при этом обычные клиенты, например Mozilla/5.0, продолжают работать без ограничений. Настроенное правило обеспечивает дополнительную L7-защиту от подозрительных сканеров уязвимостей.

Задание 3. Rate Limiting
Что делали: запустили стресс-тест целевого эндпоинта до и после настройки механизма ограничения интенсивности запросов (Rate Limiting). Для варианта 4 выполнялась защита эндпоинта /api/password_reset с параметрами:

- rate = 1r/s
- burst = 2

Что ввели в nginx/user_nginx.conf:

```
limit_req_zone $binary_remote_addr zone=mylimit:10m rate=1r/s;

server {
    listen 80;
    server_name localhost;

    location /api/password_reset {
        limit_req zone=mylimit burst=2 nodelay;
        proxy_pass ${BACKEND};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location / {
        proxy_pass ${BACKEND};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Что вводили в терминал:

ДО Rate Limiting

.\hey.exe -n 500 -c 10 -U "Mozilla/5.0" -H "Content-Type: application/json" http://127.0.0.1/api/password_reset

Перезапуск WAF

docker compose restart waf

ПОСЛЕ Rate Limiting

.\hey.exe -n 500 -c 10 -U "Mozilla/5.0" -H "Content-Type: application/json"
http://127.0.0.1/api/password_reset

Контрольный одиночный запрос

curl.exe -i
http://127.0.0.1/api/password_reset

Что получили:

Тест	Распределение кодов
ДО (без лимита)	[200] 500 responses — все запросы успешны
ПОСЛЕ (с лимитом 1r/s, burst=2)	[200] несколько успешных запросов + [503] большая часть запросов заблокирована
Одиночный запрос	200 OK — запрос проходит, так как соответствует установленному ограничению

Лог WAF:

[error] limiting requests, excess: 2.900 by zone "mylimit", client: 172.18.0.1

Вывод: Директивы limit_req_zone и limit_req с параметрами rate=1r/s и burst=2 успешно ограничивают интенсивность запросов к эндпоинту /api/password_reset. При превышении установленного лимита сервер начинает возвращать код 503 Service Unavailable, что предотвращает выполнение L7 DDoS-атак и массовых автоматизированных запросов. Одиночные легитимные запросы продолжают обрабатываться корректно.

Задание 4. Проектирование профилей защиты

4.1. Защита от Slowloris

Конфигурация Nginx:

client_header_timeout 10s;

client_body_timeout 10s;

keepalive_timeout 15s;

send_timeout 10s;

Обоснование:

Директива	Значение	Обоснование
client_header_timeout	10 с	Ограничивает время ожидания получения HTTP-заголовков от клиента. Обычный пользователь передаёт заголовки практически мгновенно, поэтому 10 секунд достаточно для медленных соединений, но недостаточно для атаки Slowloris.
client_body_timeout	10 с	Определяет максимальное время ожидания передачи тела запроса. Позволяет предотвратить атаки типа slow POST, когда злоумышленник удерживает соединение длительной отправкой данных.
keepalive_timeout	15 с	Ограничивает время хранения неактивного соединения. Большие значения могут позволить атакующему удерживать большое количество открытых подключений и расходовать ресурсы сервера.
send_timeout	10 с	Разрывает соединение, если клиент слишком долго не получает ответ от сервера. Используется для защиты от атак slow read.

Принцип работы защиты заключается в том, что атака **Slowloris** создаёт большое количество соединений и отправляет HTTP-заголовки с очень низкой скоростью, постепенно занимая доступные соединения сервера. Установка небольших таймаутов позволяет автоматически закрывать подобные соединения и освобождать ресурсы.

Дополнительно:

limit_conn_zone $binary_remote_addr zone=addr:10m;
limit_conn addr 20; не более 20 соединений с одного IP
limit_rate 500k; ограничение скорости отдачи

4.2. Эшелонированная защита CDN + WAF + Backend
Схема обработки трафика:

Роли слоёв:

[Пользователь]
↓
[CDN: L3/L4 защита, Anycast, кэш, скрытие origin-IP]
↓
[WAF: ModSecurity + OWASP CRS, L7 защита, Rate Limiting]
↓
[Backend: Flask-приложение]

1. CDN - распределённая сеть edge-серверов:
    - Anycast-маршрутизация - трафик атаки «размазывается» по сотням точек, ни один узел не перегружается
    - Кэширование статики - снижает нагрузку на origin
    - Фильтрация L3/L4 - SYN-flood, UDP-flood, amplification гасятся на периферии
    - Скрытие реального IP сервера приложения
2. WAF - анализ L7-трафика:

    - Сигнатурный анализ OWASP CRS (SQLi, XSS, RFI, LFI, RCE)
    - Пользовательские правила (блокировка специфичных UA)
    - Rate Limiting (защита от L7 DDoS и брутфорса)
    - Виртуальный патчинг уязвимостей
3. Backend - конечная точка, получает только валидные HTTP-запросы.

Ключевой принцип - defense in depth: CDN берёт на себя объём и L3/L4, WAF - сигнатуры и логику L7. Если атакующий пробьёт один слой, второй его остановит.

Вывод: В ходе выполнения задания была рассмотрена архитектура многоуровневой защиты веб-приложений. Использование CDN позволяет снизить нагрузку от сетевых атак L3/L4, а WAF обеспечивает анализ и фильтрацию запросов на уровне приложения L7. Дополнительная настройка ограничений Nginx позволяет эффективно противодействовать DDoS-атакам и массовым автоматизированным запросам.